JAWVEE

LLM-Free Handwriting Intelligence Engine

A technical design for real-time digital-ink recognition, segmentation, writing assessment, and teaching children how to write — without requiring an LLM in the recognition loop.

Scope. This document covers only the handwriting/ink intelligence subsystem. The central question is: how can JAWVEE understand what a child writes immediately, distinguish a small A from a large A, and distinguish one character from a word, line, paragraph, or drawing?

Executive thesis: do not build this as screenshot OCR. Build it as an **online digital-ink engine**. The device supplies time-ordered points and stroke boundaries; a dedicated local recognition/layout stack turns those strokes into semantic units.

Research grounding includes Microsoft's Ink Analysis architecture, IAM On-Line Handwriting Database, UNIPEN, CROHME, and research on online handwriting recognition. These sources support the separation of handwriting recognition from layout analysis and the use of hierarchical ink structures.

1. The Core Idea

The concept is technically feasible and is a strong fit for JAWVEE. The important refinement is that the canvas itself does not “understand A.” The touch/stylus layer supplies raw contact data; the handwriting engine above it interprets that data.

The system should operate without an LLM for low-level perception. A child writes A, and the local engine can produce something like A = 0.97 confidence. The LLM is unnecessary for that operation.

The same principle should apply to numbers, basic mathematical symbols, and eventually richer mathematical/scientific ink.

```
Raw touch/stylus points
        ↓
Stroke collector
        ↓
Normalization
        ↓
Writing-vs-drawing classification
        ↓
Layout / segmentation
region → paragraph → line → word → character
        ↓
Local recognizer
        ↓
Structural / formation verifier
        ↓
Semantic ink graph
        ↓
Teaching / assessment layer
```

2. Why Online Ink Is Better Than a Screenshot

When the child writes on a digital canvas, JAWVEE can receive a time-ordered trajectory. A stroke is a sequence of x/y points with timestamps while contact is down. Pen-up ends the stroke. Optional stylus data can include pressure and tilt.

That means the system has information a screenshot does not: stroke order, timing, velocity, pauses, and exact trajectories. Online-handwriting research specifically identifies time-ordered stroke sequences as a major advantage over offline recognition from scanned images. UNIPEN likewise represents pen trajectories using coordinate/time data and allows additional channels and segmentation annotations. [Sources: online-recognition survey; UNIPEN.]

```
Stroke = {
  id,
  points: [{x,y,t,pressure?,tilt?}, ...],
  startTime,
  endTime,
  bounds
}

InkDocument = {
  strokes: [...],
  writingRegions: [...],
  layoutHypotheses: [...],
  semanticUnits: [...]
}
```

3. The Critical Insight: Recognition and Segmentation Are Different Problems

This is the most important part of the design.

“Is this an A?” is a recognition problem.

“Which strokes belong to the A?” is a segmentation problem.

“Are these five As, one word, a line, or part of a paragraph?” is a layout problem.

A robust system therefore needs a hierarchy rather than one giant classifier. Microsoft’s Ink Analysis documentation explicitly describes the combination of handwriting recognition and layout analysis, including recognition segments, words,

lines, paragraphs, drawings, and incremental analysis. [Source: Microsoft Ink Analysis.]

```
Ink
 ■■■ Region
 ■    ■■■ Paragraph
 ■    ■    ■■■ Line
 ■    ■    ■    ■■■ Word
 ■    ■    ■    ■    ■■■ Character / symbol
 ■    ■    ■    ■    ■■■ Character / symbol
 ■    ■    ■    ■■■ Word
 ■    ■    ■■■ Line
 ■    ■■■ Drawing
 ■■■ Unknown / ambiguous
```

4. Solving the Big-A / Small-A Problem

Never use absolute screen pixels as the primary determinant of character identity.

For each candidate ink unit, calculate its bounding box and normalize the trajectory into a local coordinate system. A 30×40 pixel A and a 300×400 pixel A can therefore produce essentially the same recognition representation.

However, preserve the original geometry. Recognition uses normalized geometry; educational assessment can still use original size, placement, baseline, and spacing.

```
xmin = min(x); xmax = max(x)
ymin = min(y); ymax = max(y)

w = xmax - xmin
h = ymax - ymin

nx = (x - xmin) / w
ny = (y - ymin) / h

Recognition → normalized shape
Assessment   → original geometry
```

5. How the System Knows It Is One Letter vs a Paragraph

There is no single threshold. JAWVEE should infer structure from multiple signals:

• spatial proximity and gaps
• common baseline
• bounding boxes
• stroke timing and pen-up intervals
• relative character height
• writing direction
• local density
• expected task/unit
• recognizer confidence
• language/context when appropriate

The crucial point is that the system should use **relative geometry**. A large child may write large letters and naturally create large gaps. Thresholds based on absolute pixels will fail across devices and children.

```
Signals:
Character → local strokes + small relative extent
Word      → characters + larger relative gaps
Line      → compatible baselines + horizontal ordering
Paragraph → multiple nearby line bands
Drawing   → geometric/diagram evidence + weak text evidence
```

6. Task Context Is a Major Advantage

JAWVEE should not treat every interaction as unconstrained handwriting recognition.

If the lesson says “Write capital A” and presents one writing box, the expected unit is one character. A huge A and a tiny A are both candidate As; the system separately evaluates whether the child stayed inside the writing area.

If the lesson says “Write a sentence,” the expected unit becomes a line/sentence.

If the child is on a blank free canvas, the system uses general layout analysis.

This is a standard engineering pattern: constrained recognition is easier and more reliable than unrestricted recognition. Microsoft explicitly distinguishes constrained handwriting recognition for simple input from free-form ink analysis for richer document structure. [Source: Microsoft Windows Ink documentation.]

```
Guided mode:
  expectedUnit = CHARACTER
  expectedClass = ["A"]
  activeRegion = box_17
  maxCharacters = 1

Free-writing mode:
  expectedUnit = DOCUMENT
  classify writing/drawing
  infer line/word/character hierarchy
```

7. Stroke Collection and Real-Time Processing

The UI should never wait for recognition before drawing. Ink rendering is immediate; recognition runs alongside it.

Use a two-speed architecture:

1. Fast path: provisional recognition immediately after a stroke.
2. Commit path: stronger segmentation/recognition after pen-up or a short idle window.
3. Reconciliation path: later strokes can revise earlier segmentation.

Microsoft’s incremental-analysis documentation describes the same general principle: changed or “dirty” regions are reanalyzed while unaffected regions remain cached, allowing interactive editing without reprocessing the entire document. [Source: Microsoft Basic Recognition and Ink Analysis.]

```
PointerDown → create stroke
PointerMove → append points
PointerUp   → finalize stroke
              ↓
        provisional analysis
              ↓
        background analysis
              ↓
        commit / revise
```

8. Character Recognition

The recognizer should output a probability distribution, not only a label. A practical first version can begin with template/trajectory matching, such as DTW-style comparison, to validate the pipeline. A production version should move to a compact sequence model trained on varied handwriting.

Possible model inputs include normalized x/y trajectories, direction, velocity, curvature, stroke order, and optional pressure/tilt. The model should be small enough to run locally and quickly.

```
Input: candidate strokes

A     0.962
H     0.018
Λ     0.011
4     0.004
other 0.005

Do not immediately discard alternatives.
Keep confidence + provenance.
```

9. Character Segmentation

This is the harder part of free writing. One character can contain multiple strokes, and a child can lift the finger in the middle of a character.

Therefore, do not assume one stroke = one character.

Generate candidate stroke groups and score multiple segmentation hypotheses. Spatial gaps, baseline compatibility, stroke timing, intersections, shape likelihood, and task context should all contribute.

```
Hypothesis A:
  strokes [12,13,14] → "A"
  score = 0.91

Hypothesis B:
  strokes [12,13] → "Λ"
  stroke [14] → "-"
  score = 0.27

Keep/revise hypotheses until sufficient evidence exists.
```

10. Word Segmentation

Word boundaries can be estimated from relative horizontal gaps.

Instead of a fixed “20 pixels means a new word” rule, estimate the child’s current character scale and compute gaps relative to local character width/height. Better still, model every gap as a boundary probability.

This automatically adapts when one child writes large and another writes small.

```
medianCharacterWidth = median(confident character widths)

gapScore =
  f(gap / medianCharacterWidth,
    baselineCompatibility,
    strokeTiming,
    languageContext)

P(newWord | gap_17) = 0.82
```

11. Line and Paragraph Segmentation

Line detection is mainly a spatial-layout problem. Estimate a baseline for candidate writing units, cluster units with similar baselines, then order them horizontally.

A new line is supported by a sufficiently large vertical separation relative to local character height. A paragraph is a group of compatible lines in the same writing region.

Do not use fixed screen y-coordinates: a child may write on a slant, the page may scroll, or the canvas may zoom.

```
Line 1 baseline ≈ 120
Line 2 baseline ≈ 185

character height ≈ 32
relative separation ≈ 65 / 32 = 2.03

→ strong evidence for a new line
```

12. Writing vs Drawing

JAWVEE’s canvas will contain handwriting, AI drawings, circles, arrows, graphs, diagrams, and scribbles. The handwriting engine must not interpret every stroke as text.

Use a classifier/hypothesis layer combining task context, text-likeness, shape recognition, baseline evidence, proximity, known diagram objects, and recognizer confidence.

Keep an “unknown” state. A low-confidence stroke group should not be permanently committed as text.

Microsoft’s Ink Analysis architecture explicitly supports writing-vs-drawing classification alongside text, shape, and layout analysis. [Sources: Microsoft Ink Analysis.]

```
Ink group
  ■■ text likelihood
  ■■ drawing likelihood
  ■■ shape likelihood
  ■■ task-context likelihood
  ■■ semantic-scene compatibility

→ writing / drawing / ambiguous
```

13. Writing Assessment Is Separate From Recognition

A system can be highly confident that a child wrote A while the formation quality is poor. Those are different measurements.

JAWVEE should score symbol identity separately from formation quality.

```
Identity        0.97
Stroke order    0.54
Direction       0.72
Shape           0.89
Crossbar        0.31
Baseline        0.86
Size/placement  0.77

Recognition: A
Teaching diagnosis: crossbar formation
```

14. What to Measure When Teaching Writing

Useful dimensions include:

- symbol identity
- stroke count
- intended stroke order
- stroke direction
- shape completeness
- intersections/closure
- proportions
- baseline alignment
- height and width
- spacing
- hesitation/fluency
- placement inside a target region

Do not require pixel-perfect imitation. The tutor should optimize for legibility, age-appropriate formation, and progressive motor skill—not one exact font.

```
Child writes A
      ↓
Recognize A
      ↓
Compare formation against rubric
      ↓
Find highest-impact weakness
      ↓
Give one small correction
      ↓
Animate correct stroke if useful
      ↓
Retry
      ↓
Measure improvement
```

15. The Writing Tutor Loop

The educational product becomes much more powerful when recognition is closed-loop.

Example: JAWVEE recognizes A with high confidence but detects that the crossbar is missing. It should not say “wrong.” It should identify the specific formation issue, demonstrate the missing stroke, and ask for a retry.

The important architectural separation is:
recognition = what did the child produce?
assessment = how well was it formed?
pedagogy = what should the child do next?

```
Detected: A
Identity confidence: 0.97
Shape: 0.89
Crossbar: 0.31

Feedback:
"Nice! You made both sides of A.
Now add the small line across the middle."

Next:
animate crossbar → retry → reassess
```

16. What Does Not Need an LLM

The following should be local/dedicated deterministic or ML components:

• point collection
• stroke boundaries
• normalization
• bounding boxes
• velocity/curvature
• baseline estimation
• stroke clustering
• writing-vs-drawing classification
• constrained A–Z/0–9 recognition
• stroke-order scoring
• placement/size measurement
• line/word/paragraph layout
• stroke-to-semantic-object mapping

The LLM can be used above this layer for explanations, personalized teaching language, broader lesson reasoning, and natural-language interaction. It should not be the touch-input engine.

```
Operation                        LLM?
Capture x/y/t                     NO
Normalize trajectory              NO
Recognize A in guided mode        NO
Assess stroke order               NO
Segment line/word/paragraph       NO
Explain mistake naturally         MAYBE
Generate personalized coaching    MAYBE
Reason about broader lesson       YES
```

17. Recommended Data Model

Every recognized unit should retain provenance and its relationship to raw strokes. This is essential for debugging, replay, retraining, and educational analytics.

```
{
  "unitId": "char_1842",
  "kind": "character",
  "source": "student",
  "strokeIds": ["s41","s42","s43"],
  "geometry": {
    "bbox": [x,y,w,h],
    "baseline": 0.82,
    "normalized": [...]
```

```
  },
  "recognition": {
    "candidates": [
      {"label":"A","confidence":0.97},
      {"label":"H","confidence":0.02}
    ],
    "committed":"A"
  },
  "formation": {
    "strokeOrder":0.91,
    "direction":0.88,
    "shape":0.94,
    "placement":0.86
  },
  "layout": {
    "wordId":"word_91",
    "lineId":"line_8",
    "paragraphId":"para_2"
  }
}
```

18. Semantic Ink Hierarchy

The rest of JAWVEE should consume semantic ink objects rather than raw point arrays. A character belongs to a word; a word belongs to a line; a line belongs to a paragraph. A drawing can sit beside the hierarchy without being forced into it.

This mirrors established digital-ink analysis structures and is also valuable for future math/science ink. CROHME, for example, provides online mathematical traces with symbol-level segmentation and expression-level structure. [Source: CROHME.]

```
Document
 ■■■ TextRegion
 ■    ■■■ Paragraph
 ■         ■■■ Line
 ■         ■    ■■■ Word
 ■         ■    ■    ■■■ Character
 ■         ■    ■    ■■■ Character
 ■         ■    ■■■ Word
 ■         ■■■ Line
 ■■■ Drawing / Diagram
```

19. Training Data Strategy

Public datasets are useful for bootstrapping, but a children's writing tutor needs child-specific data. The training/evaluation corpus should include different ages, dominant hands, writing styles, device types, finger vs stylus input, character sizes, and levels of writing development.

Relevant sources include IAM-OnDB for online English handwriting, IAM for offline handwriting, UNIPEN for standardized online-ink representation, and CROHME for mathematical ink. IAM-OnDB contains online forms with writer information, transcriptions, lines, words, characters, and stroke points. [Sources: IAM-OnDB; IAM; UNIPEN; CROHME.]

```
Training record:
  writerId (pseudonymous)
  ageBand
  hand
  device/input type
  language
  strokes
  segmentation
  symbol labels
  formation labels
  task context
  consent/provenance
```

20. Personalization

Do not retrain the entire recognizer for each child. Keep a general model and maintain a lightweight child profile.

The profile can contain typical character size, baseline variance, common confusions, stroke-speed distribution, and recurring formation weaknesses.

This allows JAWVEE to say “Your letters are becoming more consistent” based on longitudinal evidence without turning every child into a separate model.

```
ChildProfile:
  typicalCharacterHeight
  baselineVariance
  commonConfusions
  formationWeaknesses
  progressHistory
```

21. Confidence and Abstention

A recognition system should never silently convert uncertainty into fact.

High confidence → commit.
Medium confidence → provisional result / continue gathering context.
Low confidence → retry/clarify.
Contradictory evidence → re-segment.

Confidence should also be separated from writing quality. “I am 99% sure this is A” does not mean “this is a well-formed A.”

```
Recognition confidence ≠ formation quality

A: 0.99 confidence
Formation: 0.48

→ semantic identity is stable
→ teaching feedback is still required
```

22. Incremental Reconciliation

Suppose the child writes three strokes that initially look like one A. Then the child writes another stroke far to the right. The earlier interpretation may remain correct—or the layout may need to be revised.

Therefore, recognition objects should be provisional until the local structure stabilizes. Later ink can split, merge, or reclassify earlier units.

This is one of the strongest reasons to preserve raw strokes and maintain an evented ink graph rather than only storing final OCR text.

```
LIVE → PROVISIONAL → SEGMENTED → RECOGNIZED → ASSESSED → COMMITTED

At any stage:
  revise / split / merge / reject / delete
```

23. Latency Architecture

For a child, handwriting feedback should feel immediate. Rendering must never depend on a server round trip or LLM completion.

Recommended timing:

- ink rendering: immediate
- point/trajectory features: continuous
- provisional character: immediately after stroke
- formation assessment: shortly after unit stabilization
- word/line reconciliation: background
- paragraph/document analysis: background/incremental
- LLM explanation: only when useful

```
UI thread:
  render ink immediately
```

```
Worker/local engine:
  analyze strokes
  recognize
  assess

Teaching layer:
  receive semantic result
  decide feedback
```

24. Recommended Algorithms by Version

Start simple enough to validate the architecture, then replace components as the dataset grows.

```
V1
  smoothing + resampling
  bounding-box normalization
  template/DTW matching
  rule-based formation metrics

V2
  learned writing/drawing classifier
  compact sequence model for symbols
  probabilistic segmentation

V3
  hierarchical layout model
  multilingual character recognition
  math/science symbol recognition
  learned formation assessment
```

25. JAWVEE System Architecture

The handwriting engine should be a dedicated subsystem connected to the canvas and semantic world model. It should not be embedded inside the LLM prompt or dependent on visual screenshots.

```
CANVAS
  ↓
Raw Ink Event Stream
  ↓
Stroke Collector
  ↓
Normalizer
  ↓
Layout Intelligence
  ■■ writing/drawing
  ■■ region
  ■■ paragraph
  ■■ line
  ■■ word
  ■■ character
  ↓
Local Recognizer
  ↓
Structural + Formation Verifier
  ↓
Semantic Ink Graph
  ↓
Teaching / Assessment
```

26. Proposed Internal API

The rest of JAWVEE should interact with semantic operations rather than raw point arrays.

```
ink.startStroke(pointerEvent)
ink.appendPoint(strokeId, point)
ink.endStroke(strokeId)

layout.update(regionId)

recognition.propose(segmentId)
recognition.commit(segmentId)

assessment.evaluate(unitId, rubricId)

ink.getUnit(unitId)
ink.getAncestors(unitId)
ink.getChildren(unitId)
```

```
ink.subscribe("unitUpdated", handler)
```

27. Evaluation Framework

Character accuracy alone is insufficient. Evaluate segmentation, latency, formation assessment, and recovery.

```
Metrics
  Character accuracy
  Top-k accuracy
  Character-segmentation F1
  Word/line segmentation F1
  Writing-vs-drawing F1
  False-commit rate
  Pen-up → stable-result latency
  Formation-score agreement with teachers
  Stroke-order accuracy
  Baseline/placement accuracy
  Recovery/reconciliation success
```

28. Critical Edge Cases

The system should explicitly test:

- very large A
- very small A
- repeated A with little spacing
- multi-stroke characters
- unusually long pauses
- slanted writing
- two lines close together
- writing mixed with drawings
- child writing outside the target box
- erasing and rewriting
- overlapping annotations
- incomplete characters
- accidental touch/multi-touch
- left-handed and right-handed writing
- different device sizes and sampling rates

```
Rule:
If a new edge case breaks the system,
add a general abstraction
(segmentation signal, constraint,
representation, validator, or capability),
not a question-specific patch.
```

29. MVP Roadmap

The first version should be deliberately constrained. The goal is to prove the digital-ink architecture and educational loop before solving unrestricted handwriting.

```
Weeks 1–2:
  stroke capture + replay + normalization

Weeks 3–4:
  A–Z / 0–9 guided recognition

Weeks 5–6:
  stroke order + shape + placement assessment

Weeks 7–8:
  guided feedback + stroke animation

Weeks 9–10:
  multi-character words + line segmentation

Weeks 11–12:
  writing/drawing classifier + evaluation harness
```

30. Final Recommendation

The right abstraction for JAWVEE is not “handwriting OCR.” It is a **Digital Ink Intelligence Engine**.

The engine should know:
what strokes exist,
how they move,
which strokes belong together,
what those groups represent,
where they sit in the document,
how confidently they are recognized,
and how well the child formed them.

Scale variation is solved by normalization. The distinction between character/word/line/paragraph is solved by hierarchical layout analysis. Writing-vs-drawing is solved by classification and context. Symbol identity is solved by a dedicated local recognizer. Writing quality is solved by a separate formation-assessment layer.

The LLM then sits above all of this and does what it is good at: teaching, explaining, personalizing, and reasoning over the structured result.

```
THE CORE PRINCIPLE

Do not ask:
“What does this whole canvas mean?”

Continuously maintain:
“Here are the strokes, here is their hierarchy,
here is what each unit probably represents,
and here is how confident we are.”

That is the foundation for a real-time JAWVEE handwriting tutor.
```

Appendix — Research Sources

1. Microsoft Learn, **Ink Analysis Overview**: describes handwriting recognition plus ink classification/layout analysis, including words, lines, paragraphs, drawings, and incremental analysis. ■cite■turn1search0■

2. Microsoft Learn, **Basic Recognition and Ink Analysis**: describes background/incremental recognition, dirty-region reconciliation, and classification of ink words/drawings. ■cite■turn1search2■

3. Microsoft Learn, **Recognize Windows Ink strokes as text and shapes**: distinguishes free-form ink analysis from constrained handwriting recognition. ■cite■turn1search1■

4. Microsoft Research, **Project Ink Analysis**: describes recognition, layout structure, shape recognition, and document analysis for digital ink. ■cite■turn0search2■

5. IAM-OnDB, **IAM On-Line Handwriting Database**: online English handwriting with writer metadata, transcription, line/word/character structure, and x/y/time stroke points. ■cite■turn0search7■turn0search11■

6. IAM Handwriting Database: offline English handwriting with forms, lines, sentences, and words. ■cite■turn0search10■turn0search13■

7. UNIPEN: standardized online handwriting data exchange format with coordinate/time trajectories, optional signals, writer metadata, and segmentation annotations. ■cite■turn1search47■

8. CROHME / ICDAR: online handwritten mathematical-expression datasets containing traces plus symbol-level segmentation and expression-level structure. ■cite■turn1search5■turn1search9■

9. Survey, **Advances in online handwritten recognition in the last decades**: reviews online handwriting recognition and notes the effects of size, speed, shape, angle, and writing medium. ■cite■turn1search4■